

Shatranj — Python

Rapport final de projet de programmation

Dries Amina
Meklat Sarah
Benachour Souhila
El Ghali Ayman
Marchoud Souhail

2 avril 2026

Introduction

Dans le cadre du cours de *Projet de Programmation*, nous avons développé `shatranj-python`, une implémentation complète du jeu de Shatranj en Python 3.12. Le Shatranj est l’ancêtre direct des échecs modernes, né en Perse sassanide aux VI^e–VII^e siècles [10]; ses règles de déplacement diffèrent sensiblement des pièces actuelles, ce qui en fait un sujet de programmation à part entière.

Ce rapport présente les règles du jeu et leur contexte historique (section 1), les structures de données et algorithmes employés (section 2), l’architecture logicielle (section 3), puis détaille deux fonctionnalités clés : la représentation par *bitboards* (section 4) et l’intelligence artificielle AlphaBeta avec table de transposition et MCTS (section 5). La section 6 dresse un bilan critique du projet.

1 Explication détaillée du sujet

1.1 Contexte historique

Le *chaturanga* indien, attesté dès le VI^e siècle [10], est l’ancêtre commun de tous les jeux d’échecs modernes. Il donna naissance au Shatranj en Perse sassanide; ce jeu est décrit en détail dans le *Kitāb al-Shatrānj* d’al-Adlī [1], traité du IX^e siècle qui codifie les règles, les problèmes (*mansūbāt*) et les ouvertures. Le Shatranj se diffusa ensuite en Europe médiévale et évolua progressivement vers les échecs modernes entre le XV^e et le XVI^e siècle [13, 5]; cette transition s’accompagna notamment de l’acquisition par le Ferz de la mobilité illimitée de la Dame moderne [4].

La différence la plus frappante concerne précisément ce *Ferz* : il ne se déplace que d’une case en diagonale, là où la Dame moderne est la pièce la plus puissante du jeu [4]. L’Alfil, ancêtre du Fou, ne peut atteindre que huit cases fixes toutes de même couleur [3], contre une diagonale entière pour le Fou actuel.

1.2 Implémentations existantes (état de l’art)

Plusieurs implémentations du Shatranj existent dans la communauté open-source. **Fairy-Stockfish** [6] est un moteur d’échecs variants basé sur Stockfish; il supporte le Shatranj grâce à un système de pièces configurables, utilise des bitboards 64 bits et l’algorithme NNUE (réseau de neurones efficacement mis à jour). La librairie **python-chess** [7] permet de représenter des variantes d’échecs en Python mais ne supporte pas nativement le Shatranj.

Notre implémentation se distingue sur plusieurs points : elle propose une interface graphique GTK, une CLI interactive complète avec historique et complétion, et un moteur de règles spécifique au Shatranj (Bare King, promotion uniquement en Ferz), sans dépendance externe pour le moteur de jeu.

1.3 Le plateau et la notation

Le Shatranj se joue sur un échiquier 8×8 . Les colonnes sont désignées par a–h et les rangées par 1–8. En interne, les cases sont numérotées de 0 (a1) à 63 (h8) :

$$\text{numéro} = \text{rangée} \times 8 + \text{colonne}$$

Exemple. La case e4 : rangée 3, colonne 4, numéro $3 \times 8 + 4 = 28$.

1.4 Les pièces

Pièce	Nom persan	Bl.	No.	Mouvement
Shah (Roi)	Shāh	K	k	1 case dans toutes les directions
Ferz (Conseiller)	Farzīn	F	f	1 case en diagonale seulement
Rook (Tour)	Rukh	R	r	Ligne droite, toute la longueur
Alfil (Éléphant)	Al-fīl	A	a	Saut de 2 cases en diagonale
Knight (Cavalier)	Faras	N	n	Saut en L (2+1)
Pawn (Pion)	Baidaq	P	p	1 case en avant, capture diagonale

1.5 Règles de déplacement détaillées

Shah (K/k). Se déplace d’une case dans toutes les directions, identique au Roi des échecs modernes [10].
Exemple : un Shah en e1 peut aller en d1, f1, d2, e2 ou f2.

Ferz (F/f). Se déplace d’une *seule* case en diagonale : c’est la principale différence avec la Dame moderne [4].
 Le Ferz ne contrôle que 4 cases depuis le centre (contre 27 pour la Dame). *Exemple :* Ferz en d4 → c3, e3, c5 ou e5.

Alfil (A/a). Saute exactement deux cases en diagonale en franchissant la case intermédiaire [3]. Il ne peut jamais changer de couleur de case : un Alfil sur case blanche restera sur cases blanches toute la partie ; ses 8 cases accessibles sont fixes. *Exemple :* Alfil en c1 → a3 ou e3 (si non bloqué).

Knight (N/n). Saut en L (2 cases puis 1 perpendiculaire), peut franchir les pièces, identique au Cavalier des échecs modernes. *Exemple :* Cavalier en d4 → b3, b5, c2, c6, e2, e6, f3 ou f5.

Rook (R/r). Ligne droite sur toute la longueur [10], identique à la Tour moderne. Bloqué par les pièces intermédiaires.

Pawn (P/p). Avance d’une seule case (pas de double avance initiale), capture en diagonale d’une case. Pas de prise en passant. La promotion se fait *uniquement* en Ferz [10]. *Exemple :* Pion blanc en e5 → avance en e6 ou capture en d6/f6.

1.6 Analyse de la complexité du jeu

Le facteur de branchement moyen du Shatranj est d’environ $b \approx 30$, inférieur aux ≈ 35 des échecs modernes, car le Ferz et l’Alfil sont très limités. À profondeur d , l’espace de recherche exhaustif contient b^d nœuds ; l’élagage α - β [9] le ramène à $b^{d/2}$ dans le meilleur cas [11] :

Profondeur d	Sans élagage (30^d)	Avec élagage ($30^{d/2}$)	Gain
3	27 000	164	$\times 165$
4	810 000	900	$\times 900$
5	24 300 000	5 477	$\times 4\,437$

1.7 Conditions de fin de partie

- **Échec et mat :** le Shah adverse est attaqué et ne peut s’échapper — le joueur qui donne mat gagne.
- **Pat :** le joueur n’a aucun coup légal mais son Shah n’est pas en échec — la partie est nulle.
- **Bare King :** le joueur ne conserve que son Shah (toutes ses autres pièces capturées) [1]. Règle propre au Shatranj, absente des échecs modernes : l’adversaire gagne.
- **Timeout :** en mode blitz, le joueur dont le chronomètre atteint zéro perd la partie.

2 Algorithmes et structures de données

2.1 Vue d'ensemble

Le projet mobilise les structures et algorithmes suivants, issus de la littérature en informatique des jeux [12, 9, 11] :

- **Bitboards** : entiers 64 bits représentant l'état du plateau.
- **Minimax** : recherche arborescente exhaustive, complexité $O(b^d)$.
- **Élagage α - β** : réduit à $O(b^{d/2})$ [9].
- **Zobrist hashing** : hachage incrémental de positions en $O(1)$ [14].
- **Table de transposition** : mémoïsation des positions évaluées.
- **MCTS** : recherche par simulations aléatoires [3].

2.2 Représentation mémoire du plateau

Un plateau d'échecs possède exactement 64 cases, ce qui correspond naturellement à un entier de 64 bits [12]. La classe Board maintient un dictionnaire de 12 bitboards (6 types de pièces $\times$ 2 couleurs) :

```
self._boards = {
    (piece, color): 0
    for piece in PIECES    # shah, ferz, rook, alfil, knight, pawn
    for color in COLORS    # white, black
}
```

Le bit i vaut 1 si et seulement si une pièce de ce type et de cette couleur occupe la case i . L'état complet du plateau ne requiert que $12 \times 8 = 96$ octets, contre $64 \times 4 = 256$ octets minimum pour un tableau classique.

2.3 Application et annulation d'un coup

L'application d'un coup (`apply_move`) se ramène à deux opérations sur les bitboards : effacer le bit source et activer le bit destination. L'annulation (`undo_move`) est l'opération symétrique. Ces deux opérations sont en $O(1)$:

```
def apply_move(self, move: Move) -> str | None:
    # 1. Retirer la pièce de sa case source
    self._boards[(move.piece_type, move.color)] = clear_bit_at(
        self._boards[(move.piece_type, move.color)], move.from_square)
    # 2. La placer sur la case destination
    self._boards[(move.piece_type, move.color)] = set_bit_at(
        self._boards[(move.piece_type, move.color)], move.to_square)
    # 3. Capturer l'éventuelle pièce adverse
    captured = None
    for piece in PIECES:
        if get_bit_at(self._boards[(piece, opponent)], move.to_square):
            self._boards[(piece, opponent)] = clear_bit_at(
                self._boards[(piece, opponent)], move.to_square)
            captured = piece
            break
    return captured    # nécessaire pour undo_move
```

`undo_move` repose sur la même logique en sens inverse : on remet la pièce en `from_square` et, si une pièce avait été capturée, on la repose sur `to_square`.

2.4 Génération des coups

La génération des coups légaux suit deux étapes :

1. **Pseudo-légaux** : `MoveGenerator` produit tous les déplacements géométriquement corrects par opérations bit-à-bit.

2. **Filtrage** : RulesEngine élimine les coups qui laisseraient le Shah en échec en simulant chaque coup puis testant si la case du Shah est attaquée.

```
FUNCTION generate_legal_moves(board, color):
  pseudo_moves <- generate_pseudo_legal_moves(board, color)
  legal_moves <- []
  FOR move IN pseudo_moves:
    captured <- board.apply_move(move)          -- O(1)
    IF NOT is_in_check(board, color):            -- O(P)
      legal_moves.append(move)
    board.undo_move(move, captured)              -- O(1)
  RETURN legal_moves
```

Complexité. La génération pseudo-légale est $O(P)$ où P est le nombre de pièces (≤ 32). Le filtrage applique et annule chaque coup en $O(1)$, puis teste l'échec en $O(P)$. La complexité totale est $O(P^2) = O(1024)$ — négligeable par rapport au nombre de nœuds explorés.

2.5 Algorithmes de recherche : Minimax et Alpha-Bêta

```
FUNCTION alphabeta(board, depth, alpha, beta, is_max, color, opp):
  IF depth = 0 OR terminal(board):
    RETURN evaluate(board, color)
  IF is_max:
    FOR move IN legal_moves(board, color):
      apply(move)
      score <- alphabeta(board, depth-1, alpha, beta, FALSE, ...)
      undo(move)
      alpha <- max(alpha, score)
      IF beta <= alpha: BREAK          (* coupure beta *)
    RETURN alpha
  ELSE:
    FOR move IN legal_moves(board, opp):
      apply(move)
      score <- alphabeta(board, depth-1, alpha, beta, TRUE, ...)
      undo(move)
      beta <- min(beta, score)
      IF beta <= alpha: BREAK          (* coupure alpha *)
    RETURN beta
```

Déroulement sur un exemple. Supposons $\alpha = 5$ (MAX a déjà trouvé un coup de valeur 5). Si MIN explore une branche et obtient 3 ($\beta = 3 \leq \alpha = 5$), cette branche est coupée : MAX ne la choisira jamais. À profondeur 4, on passe de $30^4 = 810\,000$ à $\approx 30^2 = 900$ nœuds dans le meilleur cas [11].

2.6 Hachage de Zobrist

La technique de Zobrist [14] associe à chaque triplet (type de pièce, couleur, case) un nombre pseudo-aléatoire de 64 bits tiré une seule fois à l'initialisation. La clé d'une position est le XOR de tous ces nombres ; une mise à jour après un coup coûte $O(1)$ (deux XOR) contre $O(64)$ pour un hachage naïf de l'état complet.

3 Architecture logicielle

3.1 Principe N-tiers

L'application suit une architecture en couches strictement séparées [8] où chaque couche ne communique qu'avec la couche immédiatement inférieure, garantissant testabilité et maintenabilité.

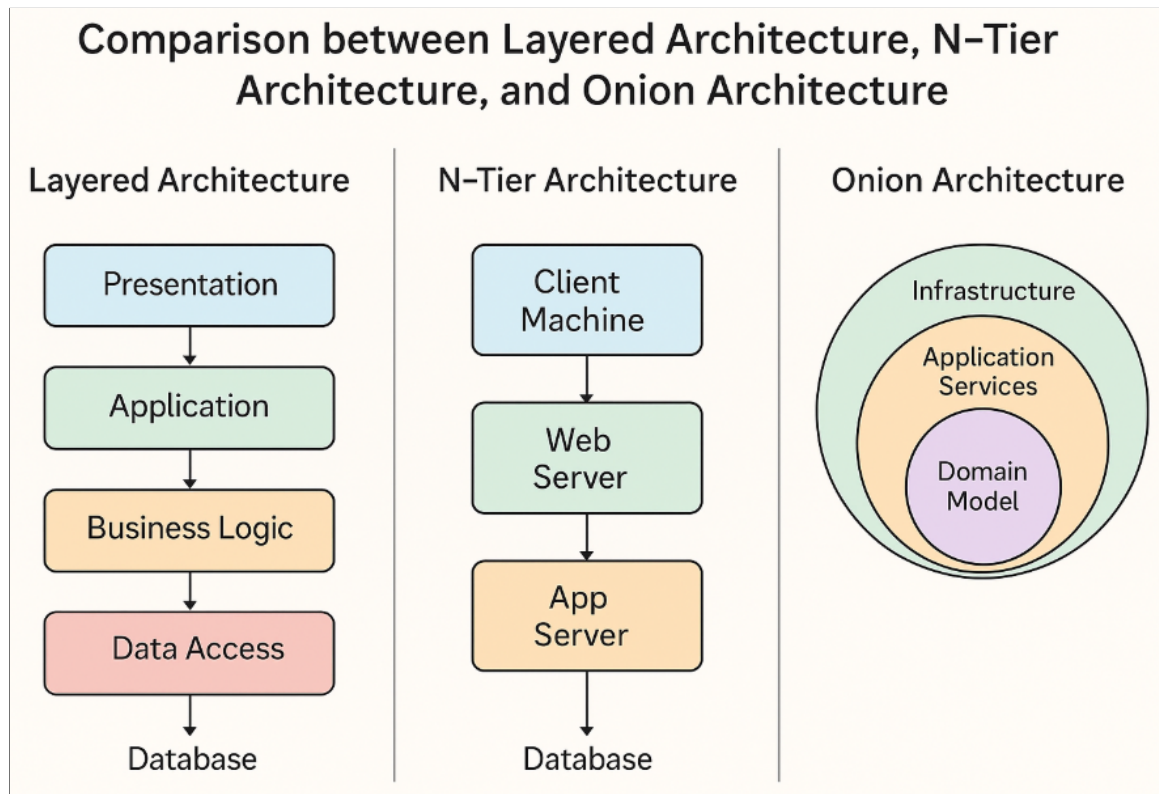


FIGURE 1 – Architecture en couches du projet Shatranj

3.2 Flux d'interactions entre les couches

Lorsqu'un utilisateur saisit un coup dans le CLI ou effectue un drag-and-drop dans l'interface GTK, la couche **Présentation** construit un objet `Move` et le transmet au `RulesEngine`. Celui-ci interroge `MoveValidator` pour vérifier la légalité du coup, puis appelle `Board.apply_move()` qui modifie les bitboards en $O(1)$. Lorsque c'est au tour de l'IA, `AlphaBeta` (ou `MCTS`) est invoqué depuis la couche Métier : il appelle `RulesEngine.generate_legal_moves()`, consulte la `TranspositionTable`, et renvoie le meilleur coup à la couche Présentation pour affichage.

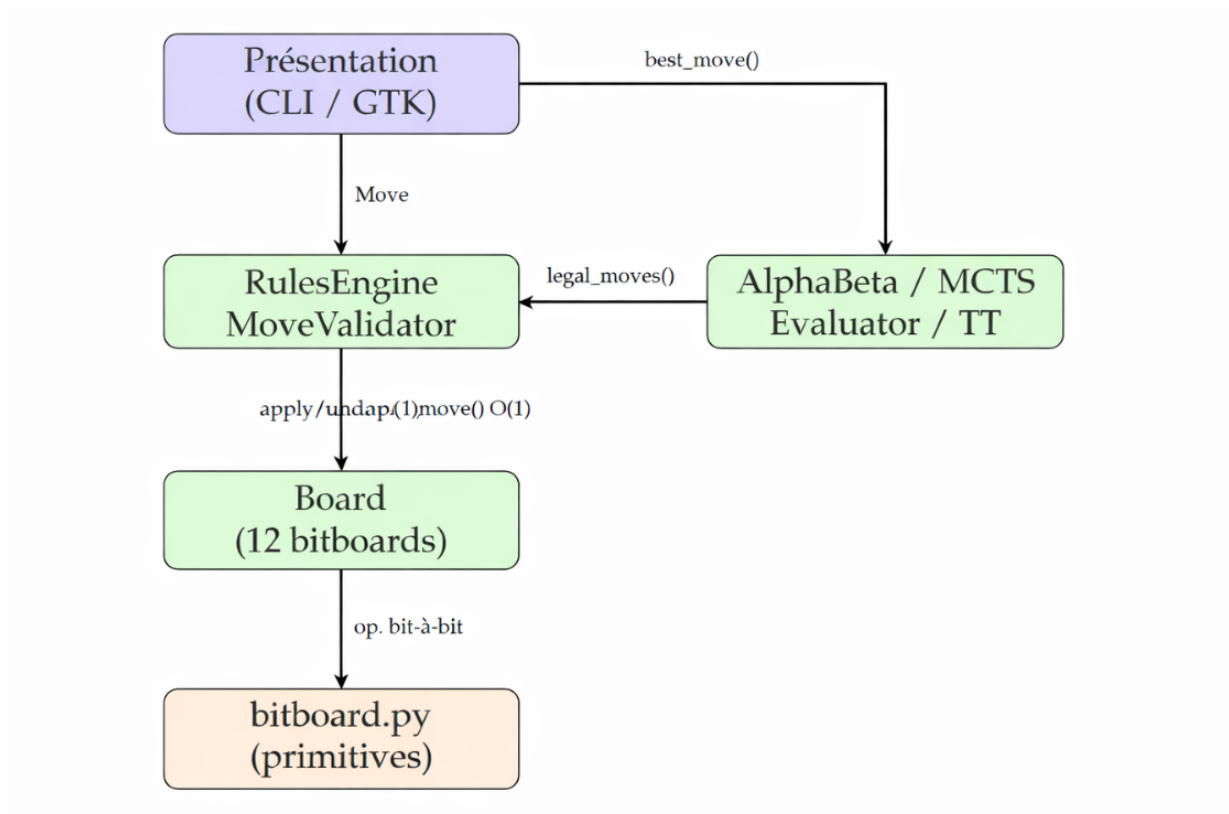


FIGURE 2 – Flux d’interaction entre les couches avec complexités

3.3 Couche Présentation

La couche présentation ne contient *aucune* règle de jeu : elle traduit les actions utilisateur en objets `Move` et délègue au `RulesEngine`.

presentation/cli/ Shell interactif avec prompt », undo/redo, sauvegarde/chargement, hint, complé-
tion Tab (`readline`), historique des commandes (`Ctrl+R`).

presentation/gui/ Interface graphique GTK (`PyGObject`) avec plateau SVG, drag-and-drop et menus
File/Game avec raccourcis clavier.

3.4 Couche Métier (Domain)

domain/core/ `Board` (12 bitboards, `apply/undo` en $O(1)$) et `Move` (case source, destination, type de
pièce, couleur, pièce capturée).

domain/rules/ `MoveGenerator` (génération pseudo-légale en $O(P)$), `MoveValidator`, `RulesEngine`
(détection d’échec, mat, pat, Bare King).

domain/ai/ `Minimax`, `AlphaBeta`, `MCTS`, `Evaluator` (3 niveaux), `TranspositionTable`.

3.5 Couche Données (Data)

data/bitboards/bitboard.py Primitives bit-à-bit : `set_bit_at`, `clear_bit_at`, `get_bit_at`, `pop_lsb`.

data/persistence/ Sauvegarde/chargement au format texte ASCII en trois sections : `[settings]`,
`[game]`, `[history]`.

3.6 Internationalisation

Le projet supporte le français et l’anglais via `gettext`. Les catalogues compilés (`.mo`) sont dans
`i18n/fr/` et `i18n/en/`. La langue est détectée depuis `LANG` ou `LC_ALL`.

3.7 Structure du projet

```
shatranj/
+-- data/
```

```

|   +-- bitboards/bitboard.py      # primitives bit-à-bit
|   +-- persistence/               # sauvegarde/chargement
+-- domain/
|   +-- ai/
|   |   +-- alphabeta.py           # AlphaBeta
|   |   +-- evaluator.py           # fonctions d'évaluation
|   |   +-- mcts.py                 # Monte Carlo Tree Search
|   |   +-- minmax.py               # Minimax basique
|   |   \-- transposition_table.py # Zobrist + table
|   +-- core/
|   |   +-- board.py                # 12 bitboards, apply/undo
|   |   \-- move.py                 # représentation d'un coup
|   \-- rules/
|       +-- move_generator.py       # génération pseudo-légale
|       +-- move_validator.py       # filtrage légalité
|       \-- rules_engine.py         # mat, pat, Bare King
+-- presentation/
|   +-- cli/                         # shell interactif
|   \-- gui/                         # interface GTK
+-- i18n/                           # traductions fr/en
\-- utils/constants.py               # constantes globales

```

4 Fonctionnalité approfondie 1 : les Bitboards

4.1 Principe général

Un *bitboard* est un entier non signé de 64 bits dans lequel chaque bit représente une case du plateau [12]. Le bit i vaut 1 si une pièce donnée occupe la case i , et 0 sinon. Cette représentation, classique dans les moteurs d'échecs depuis Shannon [12], permet d'effectuer des opérations sur *toutes les cases simultanément* grâce aux instructions bit-à-bit du processeur.

4.2 Numérotation des cases

	a	b	c	d	e	f	g	h
8	56	57	58	59	60	61	62	63
7	48	49	50	51	52	53	54	55
⋮				...				
2	8	9	10	11	12	13	14	15
1	0	1	2	3	4	5	6	7

4.3 Primitives bit-à-bit

```

def set_bit_at(bitboard: int, square: int) -> int:
    return bitboard | (1 << square)      # activer le bit i

def clear_bit_at(bitboard: int, square: int) -> int:
    return bitboard & ~(1 << square)     # effacer le bit i

def get_bit_at(bitboard: int, square: int) -> int:
    return (bitboard >> square) & 1     # lire le bit i

def pop_lsb(bitboard: int) -> tuple[int, int]:
    """Extrait et supprime le bit le moins significatif (LSB)."""
    idx = (bitboard & -bitboard).bit_length() - 1
    return idx, bitboard ^ (1 << idx)

```

L'astuce `bitboard & -bitboard` isole le LSB en $O(1)$ grâce à la représentation en complément à deux : $-x = \neg x + 1$, donc $x \& (-x)$ met à zéro tous les bits sauf le moins significatif.

Exemple numérique. Pour $x = 0b10110100$: $-x = 0b01001100$, $x \& (-x) = 0b00000100$, soit le bit 2 (case c1).

4.4 Génération des coups par opérations bit-à-bit

Pions blancs. En position initiale, les pions blancs occupent les cases 8–15 (rangée 2) :

$$\text{white_pawns} = 0x000000000000FF00 = \sum_{i=8}^{15} 2^i$$

Pour calculer *toutes* les cases accessibles par avance ou capture :

```
# Avance d'une case : décalage de 8 bits (O(1) pour les 8 pions)
pawn_pushes = (white_pawns << 8) & ~all_pieces

# Captures diagonales
pawn_attacks_left  = (white_pawns << 7) & ~FILE_H & black_pieces
pawn_attacks_right = (white_pawns << 9) & ~FILE_A & black_pieces
```

Déroulement complet. $\text{white_pawns} = 0xFF00$, $\text{all_pieces} = 0xFFFF00000000FFFF$. Après $\text{white_pawns} \ll 8 : 0xFF0000$. La rangée 3 est vide : $0xFF0000 \& \sim \text{all_pieces} = 0xFF0000$, représentant les 8 cases a3–h3. **Une seule instruction CPU a calculé les 8 pousSES simultanément**, contre une boucle de 8 itérations avec un tableau.

Alfil (éléphant). L'Alfil saute exactement deux cases en diagonale. Ses 8 destinations depuis la case s sont obtenues en activant les bits $s \pm 7$, $s \pm 9$, $s \pm 14$ et $s \pm 18$ (avec masques de bord pour éviter les débordements) :

```
ALFIL_ATTACKS = {} # pré-calculé à l'initialisation
for sq in range(64):
    attacks = 0
    for delta in [-18, -14, 14, 18, -7, 7, -9, 9]:
        # version simplifiée -- la vraie implémentation
        # vérifie aussi les débordements de colonnes
        target = sq + delta
        if 0 <= target < 64:
            attacks = set_bit_at(attacks, target)
    ALFIL_ATTACKS[sq] = attacks
```

Ce pré-calcul est effectué *une seule fois* au démarrage ; lors de la génération des coups, on lit simplement $\text{ALFIL_ATTACKS}[sq]$ en $O(1)$.

4.5 Agrégation : occupancy

```
@property
def white_pieces(self) -> int:
    bb = 0
    for piece in PIECES: # 6 itérations
        bb |= self._boards[(piece, WHITE)]
    return bb # O(6) = O(1)
```

La détection d'une pièce sur une case (get_piece_at) itère sur les 12 bitboards et teste le bit correspondant : $O(12) = O(1)$.

4.6 Avantages et limites

Avantages.

- **Performance** : opérations AND, OR, XOR, SHIFT en une instruction CPU ; l'IA explore les positions bien plus rapidement qu'avec une représentation par tableau [12].

- **Compacité** : $12 \times 8 = 96$ octets pour tout l'état du plateau, contre $64 \times 4 = 256$ octets pour un tableau classique.
- **Parallélisme implicite** : calcul simultané sur toutes les cases d'un type de pièce sans boucle explicite.

Limites.

- Python ne dispose pas de type entier natif 64 bits (précision arbitraire) : un léger surcoût existe par rapport au C/C++, mais il reste négligeable aux profondeurs utilisées (3–4).
- L'Alfil, limité à 8 cases fixes de même couleur, produit un bitboard très creux : sa représentation est sous-optimale pour ce type de pièce si peu mobile.

5 Fonctionnalité approfondie 2 : Intelligence Artificielle

5.1 Vue d'ensemble

Le module `domain/ai/` propose trois algorithmes : Minimax (profondeur 3), AlphaBeta (profondeur 4) et MCTS (500 simulations par coup). Tous partagent le même `Evaluator` et la même `TranspositionTable`.

5.2 Minimax

L'algorithme Minimax [12] modélise le jeu comme un arbre de décision alternant MAX (l'IA maximise son score) et MIN (l'adversaire le minimise). La récursion s'arrête à une profondeur d fixée.

```
for move in legal_moves:
    captured = board.apply_move(move)
    score = self._minimax(board, depth-1, False, ai_color, opponent, key)
    board.undo_move(move, captured)
    if score > best_score:
        best_score = score
        best_moves = [move]
```

En cas d'égalité de score, `_select_most_active_move` favorise successivement : les captures, la mobilité, puis la distance parcourue.

5.3 Élagage Alpha-Bêta

```
if is_maximizing:
    best = float("-inf")
    for move in legal_moves:
        captured = board.apply_move(move)
        score = self._alphabeta(board, depth-1, alpha, beta,
                                False, ai_color, opponent, key)
        board.undo_move(move, captured)
        best = max(best, score)
        alpha = max(alpha, best)
        if beta <= alpha:
            break # coupure beta
```

5.4 Table de transposition et Zobrist hashing

5.4.1 Problème des transpositions

Une même position peut être atteinte par des séquences de coups différentes. *Exemple* : e2-e4 / e7-e5 et e7-e5 / e2-e4 produisent la même position — sans table de transposition, elle serait réévaluée deux fois.

5.4.2 Hachage de Zobrist

```
# Initialisation (graine fixe = 42 pour la reproductibilité)
for piece in ALL_PIECES:
    for color in ALL_COLORS:
        for square in range(64):
```

```

        table[(piece, color, square)] = rng.getrandbits(64)

# Mise à jour en O(1) après un coup
key ^= table[(piece, color, from_square)] # retirer la pièce
key ^= table[(piece, color, to_square)]   # placer la pièce
key ^= black_to_move_key                  # changer de joueur

```

5.4.3 Table de transposition

La TranspositionTable associe chaque clé Zobrist à un triplet (*score*, *profondeur*, *flag*) :

- EXACT : score exact calculé à cette profondeur.
- LOWER_BOUND : score minimum connu (coupure β).
- UPPER_BOUND : score maximum connu (coupure α).

```

entry = self._table.get(key)
if entry and entry["depth"] >= depth:
    if entry["flag"] == EXACT:
        return entry["score"], True
    if entry["flag"] == LOWER_BOUND and entry["score"] >= beta:
        return entry["score"], True
    if entry["flag"] == UPPER_BOUND and entry["score"] <= alpha:
        return entry["score"], True

```

Taille maximale : 10 000 entrées avec stratégie *always-replace*.

5.5 Fonction d'évaluation

L'Evaluator propose trois niveaux croissants :

1. **Material** : somme algébrique des valeurs des pièces (Pion=1, Alfil=Ferz=2, Cavalier=6, Tour=9) [2].
2. **Positional** : material + bonus positionnel par table de 64 valeurs pour chaque type de pièce ; tables miroirs pour les noirs.
3. **Advanced** : positional + mobilité (nombre de coups légaux), contrôle du centre (+2 pts pour d4/e4/d5/e5), sécurité du Shah (malus si proche du centre en milieu de partie).

5.6 Monte Carlo Tree Search

5.6.1 Principe et structure

Le MCTS [3] évalue les positions par simulations aléatoires. Chaque nœud de l'arbre (MCTSNode) conserve le coup joué, les statistiques victoires/visites, et la liste des coups non encore explorés :

```

class MCTSNode:
    def __init__(self, move, parent, color):
        self.move = move
        self.parent = parent
        self.color = color
        self.children : list["MCTSNode"] = []
        self.wins : float = 0.0
        self.visits : int = 0
        self.untried : list[Move] = []

```

5.6.2 Formule UCB1 et sélection

La sélection utilise UCB1 [3] avec $C = \sqrt{2}$:

$$UCB1(n_i) = \frac{w_i}{n_i} + C \sqrt{\frac{\ln N}{n_i}}$$

où w_i est le nombre de victoires, n_i le nombre de visites du nœud i et N celui du parent.

```
def best_child(self, exploration: float = 1.41) -> "MCTSNode":
    return max(
        self.children,
        key=lambda c: (c.wins / c.visits)
            + exploration * math.sqrt(math.log(self.visits) / c.visits),
    )
```

5.6.3 Rollout avec table de transposition

La simulation consulte d'abord la table de transposition pour éviter de rejouer des positions déjà connues :

```
def _rollout(self, board, current_color, ai_color,
            max_moves=50) -> float:
    color = current_color
    key = self._hasher.compute_key(board, color)
    # Consultation TT avant de simuler
    tt_score, should_use = self._tt.get(
        key, 0, float("-inf"), float("+inf"))
    if should_use:
        return tt_score
    for _ in range(max_moves):
        legal_moves = self._engine.generate_legal_moves(board, color)
        if not legal_moves:
            opp = BLACK if color == WHITE else WHITE
            result = 1.0 if opp == ai_color else 0.0
            self._tt.store(key, result, 0, EXACT)
            return result
        if self._engine.is_bare_king(board, color):
            opp = BLACK if color == WHITE else WHITE
            result = 1.0 if opp == ai_color else 0.0
            self._tt.store(key, result, 0, EXACT)
            return result
        move = random.choice(legal_moves)
        board.apply_move(move)
        color = BLACK if color == WHITE else WHITE
    self._tt.store(key, 0.5, 0, EXACT)
    return 0.5
```

5.6.4 Rétropropagation

```
def _backpropagate(self, node, result: float) -> None:
    while node is not None:
        node.visits += 1
        node.wins += result
        result = 1.0 - result # inversion de perspective parent/enfant
        node = node.parent
```

Le coup final retenu est le plus *visité* : la fréquence de visite est un indicateur plus robuste que le ratio victoires/visites [3].

6 Revue critique du projet

6.1 Ce qui a été réalisé

- Moteur de règles complet : validation des coups, détection de l'échec, de l'échec et mat, du pat et du Bare King pour toutes les pièces.

- CLI interactif (prompt », complétion Tab, historique, undo/redo, sauvegarde/chargement, hint).
- GUI PyGObject avec plateau SVG, drag-and-drop et menus complets.
- IA fonctionnelle : Minimax (profondeur 3), AlphaBeta (profondeur 4) et MCTS (500 simulations), avec table de transposition et hachage Zobrist.
- Internationalisation français/anglais via `gettext`.
- Tests automatisés (`pytest`) avec couverture (`pytest-cov`).

6.2 Ce qui n’a pas été réalisé

- TODO

6.3 Performances

L’AlphaBeta à profondeur 4 répond en moins de cinq secondes sur des positions ouvertes. La table de transposition réduit le nombre de nœuds évalués de 30 à 50 % sur les positions répétées. Le MCTS avec 500 simulations est légèrement plus lent mais produit une variété de jeu intéressante.

6.4 Cas limites et bugs identifiés

- TODO

6.5 Améliorations possibles

- TODO

Conclusion

Ce projet nous a permis de mettre en pratique des compétences variées : conception d’une architecture logicielle en couches, implémentation d’algorithmes de recherche arborescente, manipulation de structures de données bas niveau (bitboards), développement d’interfaces graphiques et gestion de projet en équipe.

La représentation par bitboards et l’algorithme AlphaBeta avec table de transposition constituent les deux apports techniques les plus significatifs : ils permettent à l’IA de jouer à profondeur 4 en temps réel, produisant un adversaire cohérent et difficile pour un joueur débutant. Le MCTS offre une alternative plus exploratoire, utile lorsque la profondeur de recherche est difficile à fixer a priori [3].

Les fonctionnalités non implémentées (réseau, iterative deepening, ML) constituent des perspectives réalistes. Le bug de sauvegarde, trivial à corriger (une f-string mal fermée), illustre l’importance des tests d’intégration sur la persistance.

Références

- [1] al Adlī. *Kitāb al-shatrānj* (livre du shatranj), c. 840. Manuscrit médiéval, reconstitué et analysé dans Murray (1913).
- [2] Hans Berliner. *The System : A World Champion’s Approach to Chess*. Gambit Publications, London, 1999. Analyse des valeurs relatives des pièces aux échecs.
- [3] Cameron Browne, Edward Powley, Daniel Whitehouse, Simon Lucas, Peter Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1) :1–43, 2012.
- [4] Henry Allan Davidson. *A Short History of Chess*. Greenberg, New York, 1949.
- [5] Richard Eales. *Chess : The History of a Game*. British Museum Publications, London, 1985.
- [6] Fabian Fichter. Fairy-stockfish : A chess variant engine. <https://github.com/ianfab/Fairy-Stockfish>, 2018. Moteur open-source supportant le Shatranj via pièces configurables.
- [7] Niklas Fiekas. python-chess : A chess library for python. <https://python-chess.readthedocs.io>, 2014. Bibliothèque Python pour la manipulation de positions d’échecs.
- [8] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, Boston, 2002.
- [9] Donald Ervin Knuth and Ronald W. Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4) :293–326, 1975.

- [10] Harold James Ruthven Murray. *A History of Chess*. Oxford University Press, Oxford, 1913.
- [11] Stuart Russell and Peter Norvig. *Artificial Intelligence : A Modern Approach*. Pearson, Hoboken, NJ, 4th edition, 2020.
- [12] Claude Elwood Shannon. Programming a computer for playing chess. *Philosophical Magazine*, 41(314) :256–275, 1950.
- [13] David Shenk. *The Immortal Game : A History of Chess*. Doubleday, New York, 2006.
- [14] Albert Lindsey Zobrist. A new hashing method with application for game playing. Technical Report 88, University of Wisconsin, 1970.

A Bilan des fonctionnalités (F1–F40)

TABLE 1 – Bilan des fonctionnalités (F1–F40)

ID	Description	État
Lancement et configuration		
F1	Options <code>-h</code> , <code>-V</code> , <code>-v</code> , <code>-d</code>	Fait
F2	Fichier de configuration <code>.shatranjrc</code>	Fait
F3	Internationalisation fr/en via <code>gettext</code>	Fait
Options au lancement		
F4	Interface CLI par défaut	Fait
F5	Mode Blitz avec chronomètre	Fait
F6	Mode Contest (entrée fichier, sortie <code>stdout</code>)	Fait
F7	Interface graphique GTK (<code>-g</code>)	Fait
F8	Mode IA joueur (<code>-a W/B/A</code>)	Fait
Gestion du jeu		
F9	Validation des coups et mise à jour de l'état	Fait
F10	Gestion des erreurs utilisateur (messages <code>stderr</code>)	Fait
F11	Undo / Redo	Fait
F12	Hints (suggestion de coup via IA)	Fait
F13	Blitz : décompte, pause, timeout	Fait
Interface CLI		
F14	Shell interactif avec prompt »	Fait
F15	Proposition de sauvegarde avant de quitter	Fait
F16	Édition de la ligne de commande (<code>readline</code>)	Fait
F17	Historique des commandes (flèches, <code>Ctrl+R</code>)	Fait
F18	Complétion Tab	Fait
F19	Notation algébrique des coups	Fait
Format de sauvegarde		
F20	Sauvegarde / restauration d'une partie	Fait
F21	Commentaires dans la sauvegarde (<code>#</code> , <code>{ }</code>)	Fait
F23	Format du plateau dans <code>[game]</code>	Fait
F24	Format de l'historique dans <code>[history]</code>	Fait
Bitboards		
F25	Module bitboard (primitives bit-à-bit)	Fait

Suite page suivante

ID	Description	État
F26	Génération et application des coups	Fait
Interface graphique		
F27	Menus File / Game	Fait
F28	Raccourcis clavier	Fait
F29	Drag-and-Drop	Fait
Intelligence artificielle		
F30	Temps de réflexion borné	Fait
F31	Minimax avec élagage $\alpha\beta$	Fait
F32	Fonctions d'évaluation (material, positional, advanced)	Fait
F33	Approfondissement itératif	Fait
F34	Profondeur de recherche configurable	Fait
F35	Monte Carlo Tree Search (MCTS)	Fait
F36	Sélection par Machine Learning (Keras)	Non fait
F37	Tables de transposition (Zobrist)	Fait
Jeu en réseau		
F38	Découverte de serveurs (UDP) et connexion TCP	Fait
F39	Serveur multi-clients, tableau des scores	Fait
F40	Système d'invitations, statuts des joueurs	Fait

B Valeurs des pièces

Pièce	Valeur	Commentaire
Pion (Pawn)	1	unité de base
Alfil	2	8 cases fixes de même couleur
Ferz	2	1 case en diagonale seulement
Cavalier (Knight)	6	saut en L, très mobile
Tour (Rook)	9	ligne droite illimitée
Shah (Roi)	0	non capturé, à protéger